

Software Design

Chapter 5 — Source-Grounded Study Notes

Course: Advanced Software Engineering (CS603), 3 Credit Hours

Instructor: Asst. Prof. Dr. Ali Fahim Ni'ma

Institution: College of Computer Science & Information Technology, University of Wasit

Source: Aggarwal & Singh, Software Engineering, 3rd ed., Ch.5 companion slides (99 slides)

Method: Every paragraph anchored to its source slide as [S:n]

Status: Draft for review — verify anchors before final use

Table of Contents

Update field (right-click > Update Field / F9)

(In Word: select all and press F9, or right-click > Update Field.)

List of Figures

Figure 5.1. The design framework.

Figure 5.2. A two-part design process.

Figure 5.3. The transformation of an informal design into a detailed design.

5. Software Design

5.1 What Is Design?

The design phase is the stage of software development in which the designer plans *how* a software system is to be produced so that it is functional, reliable, and reasonably easy to understand, modify, and maintain. Its input is the software requirements specification (SRS), which states *what* the system must do; its output is a software design document (SDD), which states *how* the system will do it. Designing a software system therefore means determining how the requirements are to be realised, and the purpose of the design phase is, in one sentence, to produce a working solution to the problem posed in the requirements specification.

Design and analysis differ in both goal and scope. The goal of analysis is to elaborate the customer's requirements carefully, and to do so while deliberately avoiding any decision about the exact way the system will be implemented. An analysis model is therefore generic and platform-independent, and it is normally very difficult to implement directly. Design does the opposite: it introduces the implementation decisions and produces a model that is detailed enough to be implemented in a programming language with little further interpretation. The design model is obtained from the analysis model by a series of transformations, none of which changes the requirements, but each of which fixes more of the solution.

Design is a more creative activity than analysis, and this creativity has a consequence that surprises newcomers: there is no such thing as a single *correct* design. Analysis can, in principle, arrive at a correct model of an existing system, but good design is always system-dependent — what is good design for one system may be bad for another. Design is a problem-solving activity and, in practice, very much a matter of trial and error; the designer works together with the users, searching for a solution with professional judgement until there is agreement that a satisfactory solution has been found.

Why give design a phase of its own? The answer is scale. For a small project, such as a student exercise, one can sit with the specification and simply write the program. For a larger project, a wide gap opens between the specification and the code, and that gap has to be bridged with something more concrete. The bridge is the software design.

The design framework

Design proceeds through a framework of activities rather than in a single step (Figure 5.1). The framework starts from the initial requirements and ends with the completed design. Data is gathered on the user requirements and analysed; the questions raised by the requirements are answered, and a high-level design is then conceived. The design is refined and documented, and it is validated against the requirements on a regular basis. Because validation can raise fresh requirement questions, the activity is cyclic: the design is refined in every pass and finally documented as the software design document.

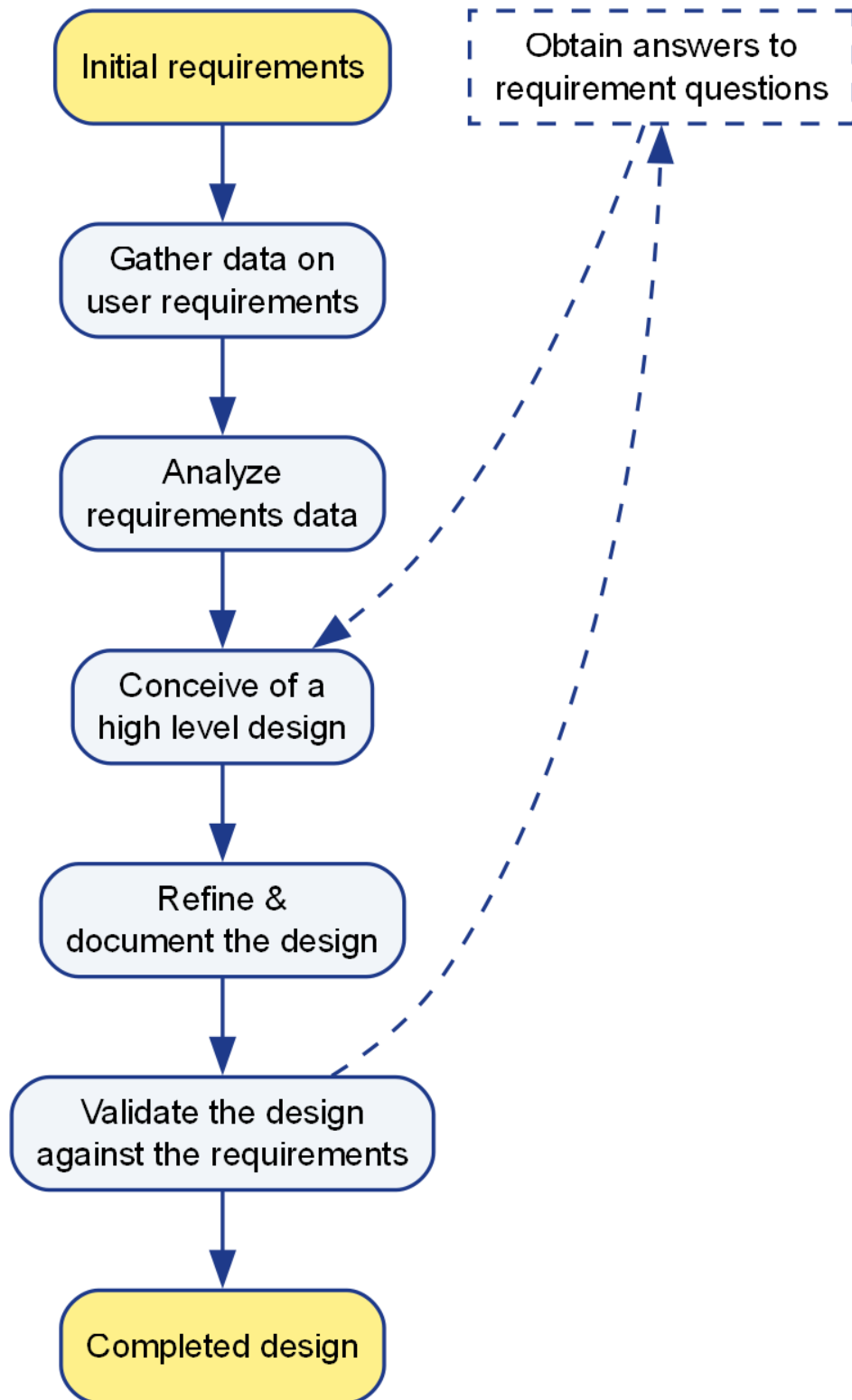


Figure 5.1. The design framework.

5.1.1 Conceptual and Technical Designs

The process of design is the transformation of ideas into detailed implementation descriptions, with the goal of satisfying the software requirements. To transform requirements into a working system, the designer must satisfy two very different audiences. The customers understand *what* the system is to do, while the system builders — the programmers who write the code — must understand *how* the system is to work. Design is therefore a two-part, iterative process.

The first part is the **conceptual design**, which tells the customer exactly what the system will do. Once the customer has approved the conceptual design, it is translated into a much more detailed document, the **technical design**, which allows the system builders to understand the actual hardware and software needed to solve the customer's problem. Figure 5.2 shows this two-part process.

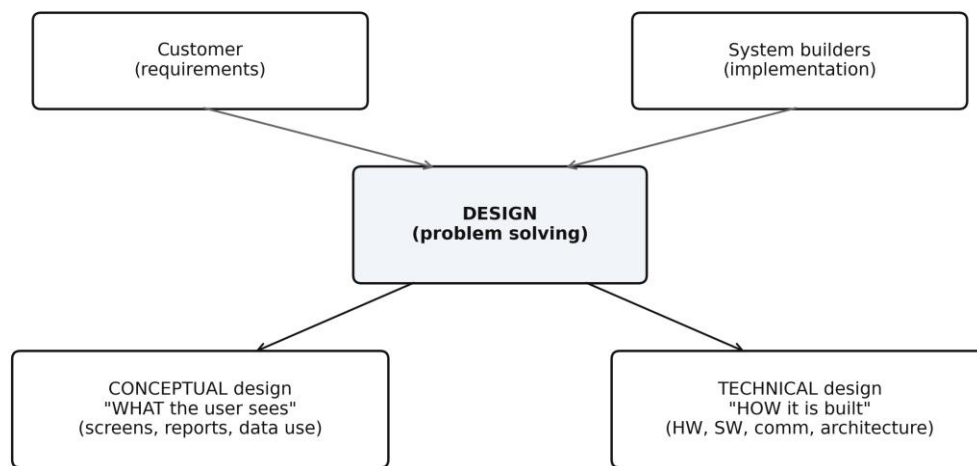


Figure 5.2. A two-part design process.

The two documents describe the same system, but in different ways, because they are written for different audiences. The conceptual design answers the following questions:

- Where will the data come from?
- What will happen to the data in the system?
- How will the system look to users?
- What choices will be offered to users?
- What is the timing of events?
- How will the reports and screens look like?

The conceptual design describes the system in a language understandable to the customer. It contains no technical jargon and is independent of any implementation. The technical design, by contrast, describes the hardware configuration, the software needs, the communication interfaces, the input and output of the system, the network architecture, and anything else that translates the requirements into a solution to the customer's problem.

The two-part split is not always necessary. Sometimes the customers are themselves software developers, and are sophisticated enough to understand the *what* and the *how* together; in such a case a single, comprehensive design document may be produced instead of two separate ones.

5.1.2 Objectives of Design

The specification — the "outside" view of a program — should be as free as possible of the aspects imposed by *how* the program will work, that is, the "inside" view. It is seldom a document from which coding can be done directly. Design therefore fills the gap between the specification and the coding: it takes the specification, decides how the program will be organized and what methods it will use, and expresses this in sufficient detail to be directly codeable. When the specification calls for a large or complex program, the design is likely to work down through a number of levels, at each level breaking the implementation problem into smaller and simpler problems. Filling a large gap involves a number of stepping-stones, and the wider the gap, the more stepping-stones are needed.

Whatever its size, the design must satisfy a set of objectives. The most widely agreed are the following. It must be **correct** — it must faithfully implement all of the functionality required by the specification. It must be **understandable**, since a design that cannot be understood cannot be implemented or maintained with confidence. It must be **efficient**, using scarce resources such as processor time and memory sensibly. It must be **maintainable**, because change requests continue to arrive long after the product is released, and because maintenance consumes the largest share of a product's lifetime cost. To these, a fuller account adds **verifiability** (the correctness of the design can be checked), **completeness** (every data structure, module, interface, and interconnection is specified), and **traceability** (every element of the design can be traced back to a requirement).

Designers do not arrive at a finished design document immediately. The design is developed iteratively through a number of phases, adding detail as it grows while constantly backtracking to correct earlier, less formal versions. The starting point is an informal design, which is refined by adding information to make it consistent and complete, passing through progressively more formal stages until the finished design is reached (Figure 5.3).

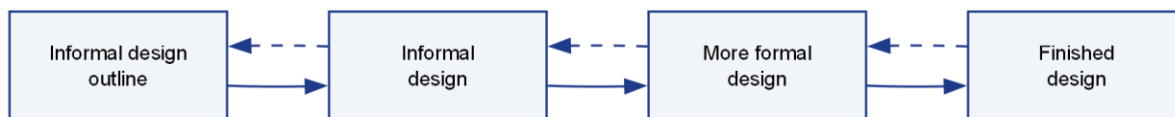


Figure 5.3. The transformation of an informal design into a detailed design.

5.1.3 Why Design Is Important

A good design is the key to a successful product. Almost two thousand years ago the Roman architect Vitruvius gave the attributes of a good design as **durability**, **utility**, and **charm**. A well-designed system is easy to implement, understandable, and reliable, and it allows the software to evolve smoothly. Without design, we risk building an unstable system: one that will fail when small changes are made, one that will be difficult to maintain, and one whose quality cannot be assessed until late in the software process. Software design must therefore contain a sufficiently complete, accurate, and precise solution to the problem to ensure that the eventual implementation is of good quality.

Three characteristics serve as a guide for the evolution of a good design:

- The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.
- The design must be a readable, understandable guide for those who generate the code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software, addressing its data, functional, and behavioural domains from an implementation perspective.

Of these, understandability deserves special emphasis. A good design exists to overcome the limits of human cognition: a large problem overwhelms the human mind, and a poor design makes the situation worse. Understandability is achieved through the disciplined use of the principles of abstraction and decomposition, so that an understandable design is one that is modular and whose modules are arranged in distinct layers. The reason the stakes are high is economic as well as technical: roughly sixty per cent of the total effort in the life cycle of a typical product is spent on maintenance, and a design that is hard to understand multiplies both development and maintenance cost while producing software that is full of defects and unreliable.